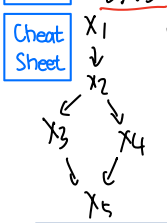
IT 5005 Cheat Sheet

BNs: Marginalisation



$P(x_2) = P(x_1)\,P(x_2|x_1)$

$P(x_2|x_3=x_3) = P(x_1)$

$P(x_2|x_1)\,P(x_3|x_2)$

$P(x_4|x_2)\,P(x_5|x_3,x_4)$

$x_1 \to x_2 \to x_3 \to x_4$

$P(x_2|x_4=x_4) =$

$P(x_1)\,P(x_2|x_1)$

$P(x_3|x_2)$

$P(x_4=x_4|x_3)$

MCs: Assumption (first order)

1. MCs (of order n): Pr of outcome depends on the previous n states
2. Stationarity (Assumed): Transition Prs same

$P(X_t|X_{t-1},X_{t-2},\dots,X_1) = P(X_t|X_{t-1})\ \forall k$

$P(X_t|X_{t-1}) = P(X_k|X_{k-1})$

MC in NLP

Unigram $w_1\ w_2\ w_3$

Digram $w_1 \to w_2 \to w_3$

Trigram $w_1 \to w_2 \to w_3$

$P(w_{123}) = P(w_1)\,P(w_2|w_1)\,P(w_3|w_1,w_2)$

MCs are BNs

Semantics apply.

Local: Cond. indep. $X \perp Y \mid Z$

$P(x|y,z) = P(x|z)$. $P(x|z) = P(x|z)\,P(y|z)$. Markov blanket: parents + childs + children's parents.

d-separation: (Global) $x \to z \to y$ d-sep by z. $x \perp y \mid z$

Joint distri.

$X_1 \to X_2 \to \cdots \to X_t$

$P(\dots) = P(x_1)\prod_{i=2}^{t} P(x_i|x_{i-1})$

Thus only need $\cdot x_1$ $\cdot$ tran. Pr.

(Cond. prob. table, CPT)

Representation eg stock

$V_1 \to V_2 \to \cdots$

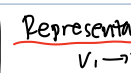
① CPT

v_{t-1} | v_t | $P(v_t|v_{t-1})$

② State Tr. Matrix

$T = \begin{bmatrix} h|h & l|h \\ h|l & l|l \end{bmatrix}$

③ State Tr. Diagram



④ Trellis'

Prediction

$P(X_t) = T^T P(X_{t-1}) = \cdots = (T^T)^t P(X_1)$

Stationary distri.

$P(X_\infty)$ or $P_\infty(X)$

$= T^T P(X_\infty)$

i.e. evector of T^T

Example

$P_\infty(\text{high}) = P(h|h)P(h) + P(h|l)P(l) = 0.9P(h) + 0.1P(l)$, $P_\infty(low) = 1 - P_\infty(high)$

$(\Sigma_v P_\infty(v) = 1)$

Application (NNC)

Diffusion Model

Page Rank

HMM as BNs

by factoring add. evidence

$S = \{1,2,\dots,|S|\}$: set of states

$x_t \in S$, O: output, $E_t \in O$

set of output symbols

HMM

X_t: State, E_t: observation

$E_{1:T}$: from 1 to T

$X_1 \to X_2 \to X_3$: hidden/unobserved states

$E_1\ E_2\ E_3$: evidence/emissions

$T_{ij} = P(X_t = j \mid X_{t-1} = i)$ Trans. prob.

$O_{ij} = P(E_t = o \mid X_t = i)$ prob. of output o in state i.

Ex. NLP:

S = parts of speech

O = vocab

π_i = init. prob.

HMM stock ex.

S: stock value $x \in S = \{h, l\}$

O: Buy or sell $E \in O = \{b, s\}$

$X_1 \to X_2 \to \cdots$

$E_1\ E_2\ \cdots$

Joint pd.

START STOP

$P(E_1,\dots,E_n, X_0, X_1, \dots, X_{n+1}) = P(x_0)\prod_{i=1}^{n+1} P(x_i|x_{i-1})$

$P(x_0)$: init. prob. $P(x_i|x_{i-1})$: trans. dist.

$\prod_{i=1}^{n} P(E_i|x_i)$: emission dist.

Interencing

Filtering: $P(X_{t+1}|e_{1:t+1})$ — estimation of today's stock based on sell/buy info. till today

Prediction: $P(X_{t+k}|e_{1:t+1})$ — esti. of future

Smoothing: $P(X_{t+1}|e_{1:N})$

Most likely explanation: $\arg\max_{x_{1:N}} P(X_{1:N}|e_{1:N})$ (Viterbi)

$O_{t+1} = \text{diag}(P(e_{t+1}|X_{t+1}=1), \dots, P(e_{t+1}|X_{t+1}=|S|))$ (observ. matrix)

$T = \begin{bmatrix} T_{11} & \cdots & T_{1|S|} \\ \vdots & \ddots & \vdots \\ T_{|S|1} & \cdots & T_{|S||S|} \end{bmatrix}$ $T_{ij} = P(X_{t+1}=j \mid X_t = i)$

Bayesian Recursive Filtering

Calculate $B(X_{t+1}) = P(X_{t+1}|e_{1:t+1})$

i) Predict $B'(X_{t+1}) = P(X_{t+1}|e_{1:t})$

ii) Update with e_{t+1}

$f_{1:t+1} = \alpha O_{t+1} T^T f_{1:t}$ (normalisation $\alpha = \frac{1}{P(e_{t+1}|e_{1:t})}$)

$P(X_{t+1}|e_{1:t+1}) = \alpha P(e_{t+1}|X_{t+1}) P(X_{t+1}|e_{1:t})$

$P(X_{t+1}|e_{1:t+1}) = \alpha P(e_{t+1}|X_{t+1}) \sum_{x_t} P(X_{t+1}|X_t=x_t) P(X_t=x_t|e_{1:t})$ (sensor, trans.)

$f_{1:t} = \begin{bmatrix} B(X_t=1) \\ \vdots \\ B(X_t=|S|) \end{bmatrix}$ (belief) FORWARD recursion

$\begin{bmatrix} B(X_{t+1}=1) \\ B(X_{t+1}=2) \end{bmatrix} = \alpha \begin{bmatrix} O_{1,1} & 0 \\ 0 & O_{2,2} \end{bmatrix} \begin{bmatrix} T_{1,1} & T_{2,1} \\ T_{1,2} & T_{2,2} \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \end{bmatrix}$

Lin. model

$x^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix}$ ← bias

$\hat{y}_j = g(w^T x^{(i)})$ for bias

$w = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}$ weight

Lin. reg

$(L_2: \|Xw - y\|^2)$

$h_w(x) = w^T x = \begin{bmatrix} w_0 \\ w_n \end{bmatrix}^T \begin{bmatrix} 1 \\ x_n \end{bmatrix}$

MSE: $J(w) = \frac{1}{2m}\sum_{i=1}^{m} J_i(w) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_w(x^{(i)}) - y^{(i)}\right)^2$

$\downarrow$ $w_0 + w_1 x^{(i)}$

Grad. desc.

$w = w - \alpha \nabla J(w)$

$w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$

$\nabla J = \begin{bmatrix} \partial J/\partial w_0 \\ \partial J/\partial w_1 \end{bmatrix}$

Batch: all data

SGD: cheaper, faster

mini-batch

$J(w) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_w(x^{(i)}) - y^{(i)}\right)^2$

$w_0 := w_0 - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(h_w(x^{(i)}) - y^{(i)}\right)\cdot x_0^{(i)}$

$\vdots$

$w_n := w_n - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(h_w(x^{(i)}) - y^{(i)}\right)\cdot x_n^{(i)}$

mean normalisation: $\frac{x_i - \mu_i}{\sigma_i}$ (or $\frac{x - \min}{\max - \min}$)

Logistic (cts.)

Threshold: $h_w(x) = \begin{cases} 1, & w^T x > 0 \\ 0, & \text{otherwise} \end{cases}$

Sigmoid (Logistic): $h_w(x) = g(w^T x)$

$g(z) = \frac{1}{1 + e^{-z}}$

NOT MSE: non-convex!

Entropy of rv: $H(P(x)) = -\sum_{x \in \text{domain}(x)} P(X=x)\log P(X=x)$

Cross-entropy: $H(P(x), P(y)) = -\sum_{c \in \text{cefa}\dots c-1} P(Y=c)\log P(Y=c)$

Losses

Huber etc. more smooth, but less sensitive. 'robust'

BCE $J_i(w) = H(P(y^{(i)}), P(\hat{y}^{(i)}))$

$= -\sum_{c \in \text{cefa},n} P(y^{(i)}=c)\log P(\hat{y}^{(i)}=c)$

= negative log-likelihood!

$J(w) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_w(x), y)$

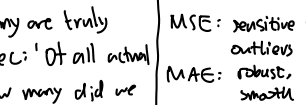
Cost

$\text{Cost}(h_w(x), y) = \begin{cases} -\log h_w(x), & y=1 \\ -\log(1 - h_w(x)), & y=0 \end{cases}$

$= -y\log h_w(x) - (1-y)\log(1 - h_w(x))$

$y=1$: $h_w(x) \to 0$, $J(w) \to \infty$

$h_w(x) \to 1$, $J(w) \to 0$



Grad. desc.

$w_n := w_n - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(h_w(x^{(i)}) - y^{(i)}\right)\cdot x_n^{(i)}$

h_w = estim. prob. that $y=1$ on input x

$(w^T x^{(i)} - y^{(i)})^2$: Widrow-Hoff

Multi-class: pick c s.t.

$\max_c h_w^{(i)}(x)$

$\text{softmax}(\vec{x}) = \left(\frac{e^{x_i}}{\sum_i e^{x_i}}\right)$

Correct = $C(\hat{y} = y)$

Accuracy = $\frac{1}{m}\sum_{j=1}^{m} C(\hat{y}_j = y_i) = \frac{TP+TN}{TP+FN+FP+TN}$

TP	FP
FN	TN

Precision = $\frac{TP}{TP+FP}$

FP: Type I

FN: Type II

Recall = $\frac{TP}{TP+FN}$ $F_1 = \left(\frac{P^{-1}+R^{-1}}{2}\right)^{-1} = \frac{2TP}{2TP+FP+FN}$

sensitivity = TPR

1-specificity = FPR

ROC, AUC. AUC = 0.5 ✗ ≈1.0 ✓

TPR = $\frac{TP}{TP+FN}$ FPR = $\frac{FP}{FP+TN}$

Normal equation

$w = (X^T X)^{-1} X^T y$ → with leading 1s

non-invertible?

- lin. dep.
- features > samples.

what to do?

- remove features
- regularisation (ridge) $(+\lambda I)$

Pre vs Rec

Pre: 'Of all predicted positives, how many are truly positive?'

Rec: 'Of all actual positives, how many did we actually detect?'

Thus, ↓FP: ↑pre. (spam, plagiarism)

↓FN: ↑rec. (cancer, disease)

Losses II

MSE: sensitive to outliers

MAE: robust, non-smooth

Huber/log-cosh: ≥0,

MSE: large, MAE

Thus, core? MSE

don't care? MAE...

Perceptron

$\hat{y} = \text{sgn}\left(\sum_{i=0}^{n} w_i x_i\right)$

PLA:

- initialise $w_i \in 0$
- $\hat{y}_i = \text{sgn}(w^T x^{(i)})$
- $w \leftarrow w + \Delta w$
- converge?

learning rate $\leftarrow \eta(y - \hat{y})x$

linearly separable if

$\exists w$: $w^T x > 0$ & $w^T x < 0$

$\exists$: converge!

$\#$: oscillate forever.

non-linear → MLP.

MLP

$z^{[L]} = W^{[L]T} a^{[L-1]} + b^{[L]}$

$a^{[L]} = g(z^{[L]})$

$W^{[L]}: n_L \times n_{L-1}$

$b^{[L]}, a^{[L]}, n_L$

Backprop: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial z^{[L]}}$, Then

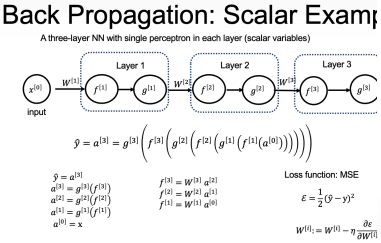
Output: $\delta^{[L]} = a^{[L]} - y$

Hidden: $\delta^{[L]} = \left(W^{[L+1]T}\delta^{[L+1]}\right) \odot g'(z^{[L]})$

$\frac{\partial \mathcal{L}}{\partial W^{[L]}} = \delta^{[L]}(a^{[L-1]})^T$, $\frac{\partial \mathcal{L}}{\partial b^{[L]}} = \delta^{[L]}$

# of param.: $n_L(n_{L-1}+1)$

Back Propagation: Scalar Exam



$\hat{y} = a^{[3]} = g^{[3]}\left(f^{[3]}\left(g^{[2]}\left(f^{[2]}\left(g^{[1]}\left(f^{[1]}(a^{[0]})\right)\right)\right)\right)\right)$

$\hat{y} = a^{[3]} = g^{[3]}(f^{[3]})$

$a^{[3]} = g^{[3]}(f^{[3]})$

$a^{[2]} = g^{[2]}(f^{[2]})$

$a^{[1]} = g^{[1]}(f^{[1]})$

$a^{[0]} = x$

$f^{[3]} = W^{[3]}\,a^{[2]}$

$f^{[2]} = W^{[2]}\,a^{[1]}$

$f^{[1]} = W^{[1]}\,a^{[0]}$

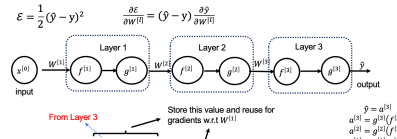
Loss function: MSE

$\mathcal{E} = \frac{1}{2}(\hat{y} - y)^2$

$W^{[l]} := W^{[l]} - \eta\frac{\partial \mathcal{E}}{\partial W^{[l]}}$

$\frac{\partial \mathcal{E}}{\partial \hat{y}} = (\hat{y} - y)$

$\frac{\partial \mathcal{E}}{\partial W^{[l]}} = \frac{\partial \mathcal{E}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial W^{[l]}}$



From Layer 3

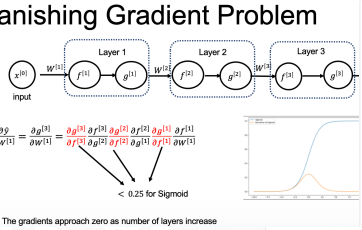
Store this value and reuse for gradients w.r.t $W^{[1]}$

$\frac{\partial \hat{y}}{\partial W^{[2]}} = \frac{\partial g^{[3]}}{\partial W^{[2]}} = \frac{\partial g^{[3]}}{\partial f^{[3]}}\frac{\partial f^{[3]}}{\partial g^{[2]}}\frac{\partial g^{[2]}}{\partial f^{[2]}}\frac{\partial f^{[2]}}{\partial W^{[2]}}$

$= \delta^{[3]}W^{[3]}\frac{\partial g^{[2]}}{\partial f^{[2]}}a^{[1]}$

$\frac{\partial \hat{y}}{\partial W^{[2]}} = \delta^{[2]}a^{[1]}$ where $\delta^{[2]} = \frac{\partial g^{[3]}}{\partial f^{[2]}} = \delta^{[3]}W^{[3]}\frac{\partial g^{[2]}}{\partial f^{[2]}}$

Vanishing Gradient Problem



$\hat{y} = a^{[3]} = g^{[3]}(f^{[3]})$

$a^{[3]} = g^{[3]}(f^{[3]})$

$a^{[2]} = g^{[2]}(f^{[2]})$

$a^{[1]} = g^{[1]}(f^{[1]})$

$f^{[3]} = W^{[3]}\,a^{[2]}$

$f^{[2]} = W^{[2]}\,a^{[1]}$

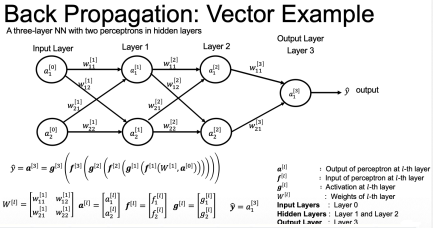
$f^{[1]} = W^{[1]}\,a^{[0]}$

$\frac{\partial \hat{y}}{\partial W^{[1]}} = \frac{\partial g^{[3]}}{\partial W^{[1]}} = \frac{\partial g^{[3]}}{\partial f^{[3]}}\frac{\partial f^{[3]}}{\partial g^{[2]}}\frac{\partial g^{[2]}}{\partial f^{[2]}}\frac{\partial f^{[2]}}{\partial g^{[1]}}\frac{\partial g^{[1]}}{\partial f^{[1]}}\frac{\partial f^{[1]}}{\partial W^{[1]}}$

The gradients approach zero as number of layers increase

Back Propagation: Vector Example

A three-layer NN with two perceptrons in hidden layers



$\hat{y} = a^{[3]} = g^{[3]}\left(f^{[3]}\left(g^{[2]}\left(f^{[2]}\left(g^{[1]}\left(f^{[1]}(W^{[1]}, a^{[0]})\right)\right)\right)\right)\right)$

$W^{[l]} = \begin{bmatrix} w_{11}^{[l]} & w_{12}^{[l]} \\ w_{21}^{[l]} & w_{22}^{[l]} \end{bmatrix}$ $a^{[l]} = \begin{bmatrix} a_1^{[l]} \\ a_2^{[l]} \end{bmatrix}$ $f^{[l]} = \begin{bmatrix} f_1^{[l]} \\ f_2^{[l]} \end{bmatrix}$ $g^{[l]} = \begin{bmatrix} g_1^{[l]} \\ g_2^{[l]} \end{bmatrix}$ $\hat{y} = a_1^{[3]}$

$a^{[l]}$: Output of perceptron at l-th layer

$f^{[l]}$: Input of perceptron at l-th layer

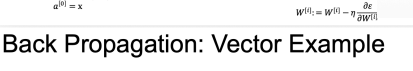
$g^{[l]}$: Activation at l-th layer

$w^{[l]}$: Weights of l-th layer

Hidden Layers : Layer 1 and Layer 2

Output Layer : Layer 3

Back Propagation: Vector Example



$\hat{y} = a^{[3]} = g^{[3]}\left(f^{[3]}\left(g^{[2]}\left(f^{[2]}\left(g^{[1]}\left(f^{[1]}(a^{[0]})\right)\right)\right)\right)\right)$

Derivatives	Recursion
$\delta^{[3]}(W^{[3]})^T$	$\delta^{[3]} = \frac{\partial g^{[3]}}{\partial f^{[3]}}$
$\delta^{[2]}(W^{[2]})^T$	$\delta^{[2]} = \frac{\partial g^{[3]}}{\partial f^{[3]}}\frac{\partial f^{[3]}}{\partial g^{[2]}}\delta^{[3]}$
$\delta^{[1]}(W^{[1]})^T$	$\delta^{[1]} = \frac{\partial g^{[3]}}{\partial f^{[3]}}\frac{\partial f^{[3]}}{\partial g^{[2]}}\frac{\partial g^{[2]}}{\partial f^{[2]}}\delta^{[2]}$

$a^{[l]} = g^{[l]}(f^{[l]})$

$f^{[l]} = (W^{[l]})^T a^{[l-1]}$

$\delta^{[l]} = \frac{\partial g^{[l]}}{\partial f^{[l]}}$

$\frac{\partial g^{[l]}}{\partial f^{[l]}}$: Diagonal Matrix

Gradients in MLP: Intuition

$\hat{y}(x) = g^{[L]}(f^{[L]}(g^{[L-1]}(\cdots(g^{[l]}(f^{[l]}(g^{[l-1]}(\cdots(g^{[1]}(f^{[1]}(x^{[0]}))))))))))$

$\delta^{[l]}$ indicates how much the output (error) of the NN will change in response to small changes in the weighted sum of inputs to the neurons at the l-th layer

$\frac{\partial g^{[L]}}{\partial f^{[l]}} = \delta^{[l]}$

$\delta'(W^{[l]}) = a^{[l-1]}\ (\delta^{[l]})^T$

$\delta^{[l]} = g'^{[l]}(f^{[l]})\ W^{[l+1]}\ \delta^{[l+1]}$ **Recursion**

$\frac{\partial g^{[l]}}{\partial f^{[l]}} = g'^{[l]}(f^{[l]})$

local gradient

Indicates how much output of a neuron changes in response to a change in weighted sum of its inputs

$\frac{\partial f^{[l+1]}}{\partial g^{[l]}}\frac{\partial g^{[L]}}{\partial f^{[l+1]}} = W^{[l+1]}\delta^{[l+1]}$

Backpropagated Error Signal

This product represents the error signal from the next layer ($l+1$) propagated backward through the weights $W^{[l+1]}$ to l-th layer

Denominator layout, i.e. by y^T and x	0	H	A^T	A	$\frac{\partial y}{\partial x}$	$\frac{\partial v}{\partial x}^T$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial x}$	$\frac{\partial u}{\partial x}A^T$	$\frac{\partial y}{\partial x}$	$\frac{\partial u}{\partial v}\frac{\partial g(u)}{\partial u}$	$\frac{\partial f(g(u))}{\partial u}\frac{\partial g}{\partial x}$
Expression	$\frac{\partial a}{\partial x} =$	$\frac{\partial x}{\partial x} =$	$\frac{\partial Ax}{\partial x} =$	$\frac{\partial x^T A}{\partial x} =$	$\frac{\partial u}{\partial x} =$	$\frac{\partial v}{\partial x} =$	$\frac{\partial a}{\partial x} =$	$\frac{\partial u}{\partial x} =$	$\frac{\partial Au}{\partial x} =$	$\frac{\partial g(u)}{\partial x} =$	$\frac{\partial f(g(u))}{\partial x} =$
Condition	a is not a function of x		A is not a function of x	A is not a function of x	$u = u(x)$, $v = v(x)$, a is not a function of x		$v = v(x)$, $u = u(x)$	A is not a function of x, $u = u(x)$		$u = u(x)$	$u = u(x)$, $v = v(x)$

Activation Functions

Activation Function	Range	Use case	
Identity: $g(x) = x$	$[-\infty, +\infty]$	Linear Regression	
ReLU: $g(x) = \begin{cases} x & if\ x > 0 \\ 0 & else \end{cases}$	$[0, +\infty]$	Linear Regression Classification	
Leaky ReLU: $g(x) = \begin{cases} x & if\ x > 0 \\ ax & else \end{cases}$	$[-\infty, +\infty]$	Linear Regression Classification	
Sigmoid: $g(x) = \sigma(x) = \frac{1}{1+\exp(-x)}$	$[0, 1]$	Linear Regression Classification	
Tanh: $g(x) = \tanh(x) = \frac{\exp(2x)-1}{\exp(2x)+1}$	$[-1, 1]$	Linear Regression Classification	
Softmax: $g(x) = \frac{\exp(-x_i)}{\sum_j \exp(-x_j)}$	$[0, 1]$	Multiclass Classification	

Loss Functions

Loss Function	Formula	Use case
Mean Square Error	$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(w^T x^{(i)} - y^{(i)})^2$	Linear Regression
Mean Absolute Error	$J(w) = \frac{1}{2m}\sum_{i=1}^{m}\lvert w^T x^{(i)} - y^{(i)}\rvert$	Linear Regression
Huber Loss	$J_\delta(w) = \begin{cases} \frac{1}{2}\sum_{i=1}^{m}(y^{(i)} - w^T x^{(i)})^2 & if\ \lvert y-\hat{y}\rvert < \delta \\ \delta\left(\sum_{i=1}^{m}\lvert y^{(i)} - w^T x^{(i)}\rvert - \frac{1}{2}\delta\right) & Otherwise \end{cases}$	Linear Regression
Tanh-Cos Loss	$J(w) = \sum_{i=1}^{m}\log(\cosh(y^{(i)} - \hat{y}^{(i)}))$	Linear Regression
Binary Cross Entropy Loss	$J(w) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)}\log\sigma(w^T x^{(i)}) + (1-y^{(i)})\log\left(1-\sigma(w^T x^{(i)})\right)$	Linear Regression Binary Classification
Multiclass Cross-Entropy Loss	$J(w) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=1}^{C} y_c^{(i)}\log\left(\hat{y}_c^{(i)}\right)$	Multiclass Classification
Hinge Loss	$J(w) = \frac{1}{m}\sum_{i=1}^{m}\max(0, 1 - y^{(i)}\hat{y}^{(i)})$	Binary Classification

Optimizers: Summary

Algorithms	Update Rule	Characteristics
Momentum-based GD	$w_{t+1} = w_t + V_{t+1}$ $V_{t+1} = \beta V_t - \alpha\frac{\partial J(w)}{\partial w}_{w=w_t}$	• Uses accumulation of past gradients to overcome local optimal point • Same learning rate for all weights is not good for convergence
AdaGrad	$w_{i,t+1} = w_{i,t} - \frac{\alpha}{\sqrt{A_{i,t}+\varepsilon}}\left(\frac{\partial J(w)}{\partial w_{i,t}}\right)$ $A_i \Leftarrow A_i + \left(\frac{\partial J(w)}{\partial w_{i,t}}\right)^2$	• Unique learning rate $\frac{\alpha}{\sqrt{A_{i,t}+\varepsilon}}$ for each weight • Learning rate is inversely proportional to accumulated gradients, i.e., $A_i + \left(\frac{\partial J(w)}{\partial w_{i,t}}\right)^2$ • Aggressive decaying of learning rate • Cannot overcome local optimal point
RMSProp	$w_{i,t+1} = w_{i,t} - \frac{\alpha}{\sqrt{A_{i,t}+\varepsilon}}\left(\frac{\partial J(w)}{\partial w_{i,t}}\right)$ $A_i \Leftarrow \beta A_i + (1-\beta)\left(\frac{\partial J(w)}{\partial w_{i,t}}\right)^2$	• Unique learning rate $\frac{\alpha}{\sqrt{A_{i,t}+\varepsilon}}$ for each feature • Learning rate is inversely proportional to exponentially weighted moving average of past gradients, i.e., $\beta A_i + (1-\beta)\left(\frac{\partial J(w)}{\partial w_{i,t}}\right)^2$ • Less aggressive decaying of learning rate which is good for convergence • Cannot overcome local optimal point
Adam (Momentum+RMSProp)	$w_{i,t+1} = w_{i,t} - \frac{\alpha_t}{\sqrt{A_{i,t}+\varepsilon}} F_i$ $F_i \Leftarrow \rho_1 F_i + (1-\rho_1)\frac{\partial J(w)}{\partial w_{i,t}}$ $A_i \Leftarrow \rho A_i + (1-\rho)\left(\frac{\partial J(w)}{\partial w_{i,t}}\right)^2$ $\alpha_t = \alpha\left(\frac{\sqrt{1-\rho^t}}{1-\rho_1^t}\right)$	• Can overcome local optima due to momentum (F_i) • Unique learning rate $\frac{\alpha_t}{\sqrt{A_{i,t}+\varepsilon}}$ for each feature • Less aggressive decaying of learning rate which is good for convergence

How to Choose Normalization Techniques?

Normalization Technique	Use Case
Data Normalization	Always
Batch Normalization	CNNs with large batches
Layer Norm, RMS Norm, PreNorm	Sequence Models and Transformers

Normalization: Why we need it?

- Stabilize and accelerate training
  - Improves conditioning
  - Enables larger learning rates with faster convergence.
- Improve gradient flow
  - Reduces vanishing/exploding gradients, allowing deeper networks
  - Make training less sensitive to weight initialization and learning rate choices.
- Decouple scale from representation
  - Makes networks robust to input scale and distribution shifts across layers

Sigmoid' max 0.25 ⇒ vanishing grad.

Low weight ⇒ slow converge

Skip connection ⇒ grad propagation from output

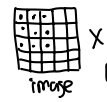
Why Layer Normalization?

- Batch-size agnostic
  - Works consistently for batch size 1 or variable batch sizes
- Ideal for sequence models
  - Sequences are usually of variable length
  - BatchNorm is NOT preferred due to dependency of batch of data
- Widely used in Transformers and RNNs,
- Stabilizes Learning by mitigating *internal covariate shift*
- Acts as a regularizer
  - Reduces the need for dropout
- Enables Higher Learning Rates
  - Stable distributions allow large learning rates
  - Speed up convergence
- Improves Gradient Flow
  - Prevents activations from becoming too large or small
  - Avoids vanishing or exploding gradients

Why batch normalization?

statistics of length

CNNs



multiply & sum (NO flip)

Output: shape H_{out} (height) $\times W_{out}$ (width) $\times k$ (# of ker)

$O = \left\lfloor \frac{I+2P-K}{S} \right\rfloor + 1$

where: I = input, O = output, P: padding. Adds zeros around the edges. $P = \frac{K-1}{2}$ to make dimensions the same.

1 2 / 3 4 —P=1→ 0 0 0 0 / 0 1 2 0 / 0 3 4 0 / 0 0 0 0

S: stride. S steps each move. S↑ → smaller feature map.

K: kernel size. (Filter = # ker) = 54 (x54 x96)

e.g. Input = 224x224 Channels: 3 (RGB) Kernels: 11x11, #=96 Stride: 4x4, P=0.

$\Rightarrow H = W = \left\lfloor \frac{I+2P-K}{S} \right\rfloor + 1 = \left\lfloor \frac{224+0-11}{4} \right\rfloor + 1$

Why CNNs? overfitting

- Spatial locality
- Parameter sharing
- Translation invariance

	MLP	vs	CNN
O	0D	(R)	2D
I	1D		3D
Layer ID			3D
CNN # of para ↓↓			

Projection mapping: input ≠ output else: identity.

Skip connection: # of filter = C_{out} Padding = 0 (to main) Stride = same Kernel size = 1x1 to ensure match.

# of parameters = ker_h * ker_w * in-channels * out-channels + # bias (for conv. layer)

where # bias = C_{out}. Σup # of par.

Pooling: → max/sum/avg. downsample.

Helps to train later kernels to detect higher-level features. Reduces dimensionality.

# of FLOPs: $2 * C_{out} * O_h * O_w * C_{in} * k_h * k_w$

# of FLOPs for maxpool: $C_{in} * O_h * O_w * (k^2 - 1)$

k = ker size. # of comparisons.

# of FLOPs per layer: $C_{out} * h_{out} * w_{out} * 2 * C_{in} * k_h * k_w$.

Transformers

RNNs/LSTMs:

- Sequential processing → cannot be parallel.
- Hard to learn long-range dependencies
- Slow inference

Tra: attention.

GPT: masked self-attention. decoder!

X cross-attention.

Input representation:

1. Token embeddings
2. Positional Encoding

$PE(p, 2i) = \sin(w_i p)$

$PE(p, 2i+1) = \cos(w_i p)$

where $w_i = \frac{1}{2i}$

p: pos of embedding vectors, i: within the vector, d: dim of model.

$PE(p) = \begin{bmatrix} \sin(w_1 p) \\ \cos(w_1 p) \\ \sin(w_2 p) \\ \vdots \\ \cos(w_{d/2} p) \end{bmatrix}$

p = pos. $w_k = \frac{1}{10000^{2k-1/d}}$

k = 1, 2, ... d/2.

$PE(p_1) \cdot PE(p_2) = \sum_{k=1}^{d/2} \cos(w_k(p_1 - p_2))$

Attention: measures similarity (relation) between vectors.

QKV: Q: 'what do I need right now?'

K: 'How should others find me?'

V: 'what info do I provide if they pick me?'

$W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$ 'differentiable' and learnable

Multi-head Attention

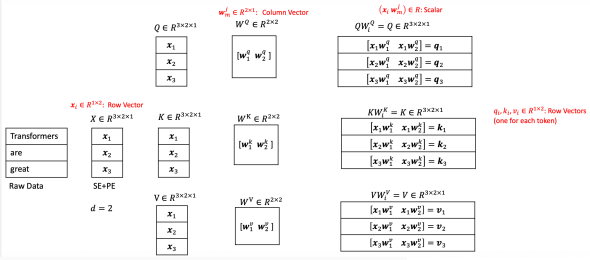
$MHA(Q, K, V) = HW_O$

where $H = concat(head_1, \dots, head_n)$, $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$.

$Att(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Masking to ensure causal property.

Scaled Dot-Product Attention: Example with Single Head



$head_i = softmax\left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_K}}\right)(VW_i^V)$

$\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}} = \begin{bmatrix} \frac{q_1 k_1^T}{\sqrt{d_1}} & \frac{q_1 k_2^T}{\sqrt{d_1}} & \frac{q_1 k_3^T}{\sqrt{d_1}} \\ \frac{q_2 k_1^T}{\sqrt{d_2}} & \frac{q_2 k_2^T}{\sqrt{d_2}} & \frac{q_2 k_3^T}{\sqrt{d_2}} \\ \frac{q_3 k_1^T}{\sqrt{d_3}} & \frac{q_3 k_2^T}{\sqrt{d_3}} & \frac{q_3 k_3^T}{\sqrt{d_3}} \end{bmatrix}$

Attention Scores

	Transformers	are	great
Transformers	a_{11}	a_{12}	a_{13}
are	a_{21}	a_{22}	a_{23}
great	a_{31}	a_{32}	a_{33}

$softmax\left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}}\right) = \begin{bmatrix} softmax\left(\left[\frac{q_1 k_1^T}{\sqrt{d_1}} \quad \frac{q_1 k_2^T}{\sqrt{d_1}} \quad \frac{q_1 k_3^T}{\sqrt{d_1}}\right]\right) \\ softmax\left(\left[\frac{q_2 k_1^T}{\sqrt{d_2}} \quad \frac{q_2 k_2^T}{\sqrt{d_2}} \quad \frac{q_2 k_3^T}{\sqrt{d_2}}\right]\right) \\ softmax\left(\left[\frac{q_3 k_1^T}{\sqrt{d_3}} \quad \frac{q_3 k_2^T}{\sqrt{d_3}} \quad \frac{q_3 k_3^T}{\sqrt{d_3}}\right]\right) \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$

Pre-LN

Learning Rate Scheduler

- Can use larger learning rate compared to post-LN
- No warmup needed

$head_i = softmax\left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_K}} \circ M\right)(VW_i^V)$ How to improve performance?

$\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}} = \begin{bmatrix} \frac{q_1 k_1^T}{\sqrt{d_1}} & -\infty & -\infty \\ \frac{q_2 k_1^T}{\sqrt{d_2}} & \frac{q_2 k_2^T}{\sqrt{d_2}} & -\infty \\ \frac{q_3 k_1^T}{\sqrt{d_3}} & \frac{q_3 k_2^T}{\sqrt{d_3}} & \frac{q_3 k_3^T}{\sqrt{d_3}} \end{bmatrix}$

Transformatoren	sind	großartig
Transformatoren	a_{11}	0	0
sind	a_{21}	a_{22}	0
großartig	a_{31}	a_{32}	a_{33}

$softmax\left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}}\right) = \begin{bmatrix} softmax\left(\left[\frac{q_1 k_1^T}{\sqrt{d_1}} \quad -\infty \quad -\infty\right]\right) \\ softmax\left(\left[\frac{q_2 k_1^T}{\sqrt{d_2}} \quad \frac{q_2 k_2^T}{\sqrt{d_2}} \quad -\infty\right]\right) \\ softmax\left(\left[\frac{q_3 k_1^T}{\sqrt{d_3}} \quad \frac{q_3 k_2^T}{\sqrt{d_3}} \quad \frac{q_3 k_3^T}{\sqrt{d_3}}\right]\right) \end{bmatrix} = \begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$

- Large d_model
- Increase sequence length (100→256), larger context
- Different activation (ReLU→GELU)
- Pre-LN
- lower drop out
- Stop the training once the validation loss ple teaus
- If validation loss increases after above changes, revert the change... it's a sign of overfitting.

Number of Parameters

- Embedding Layer: $|V| * d_{model}$. (no bias term in embedding layers)
- Positional Encoding:
  - Fixed PE: None
  - Learned PE: $L_{seq} * d_{model}$ (no bias terms)
- Per-Encoder:
  - MHA: $4 * d_{model} * d_{model} + 4 * d_{model}$ (Q, K, V, W_O matrices and 1 MHA blocks)
  - FFN: $2 * d_{model} * d_{FF} + d_{FF} + d_{model}$ ($W^{[1]}$ and $W^{[2]}$ matrices and 1 FFN block)
  - LN: $2 * 2 * d_{model}$ (γ and β and 2 LN blocks)
- Per-Decoder:
  - MHA: $2 * (4 * d_{model} * d_{model} + 4 d_{model})$ (Q, K, V, W_O matrices and 2 MHA blocks)
  - FFN: $(2 * d_{model} * d_{FF} + d_{FF} + d_{model})$ ($W^{[1]}$ and $W^{[2]}$ matrices and 1 FFN block)
  - LN: $3 * 2 * d_{model}$ (γ and β and 3 LN blocks)
- Output Layer:
  - Projection: $d_{model} * |V|$ (no bias term in output layer)

FLOPs per Layer with single head

- Attention:
  - Q, K, V Projections
    - QW^Q, KW^K, VW^V: $3 * (2 * d_{model}) * (L_{seq} * d_{model})$ (Due to multiplications and additions in matrix multiplication; FLOPs per element; # of elements)
  - Attention Scores (A)
    - $QW^Q(KW^K)^T$: $(2 * d_{model}) * (L_{seq} * L_{seq})$
    - $softmax$: $O(L_{seq}^2)$
  - Attention Values
    - $A(VW^V)$: $(2 * L_{seq}) * (L_{seq} * d_{model})$
  - Output Projection
    - $(AVW^V)W_O$: $(2 * d_{model}) * (L_{seq} * d_{model})$
- FFN
  - $(2 * d_{model}) * (L_{seq} * d_{FF}) + (2 * d_{FF}) * (L_{seq} * d_{model})$
- LN:
  - $O(L_{seq} * d_{model})$

$Q, K, V \in R^{L_{seq} \times d_{model}}$

$W^Q, W^K, W^V, W_O \in R^{d_{model} \times d_{model}}$

$QW^Q, KW^K, VW^V \in R^{L_{seq} \times d_{model}}$

$QW^Q(KW^K)^T \in R^{L_{seq} \times L_{seq}}$

$A \in R^{L_{seq} \times L_{seq}}$: Attention Matrix

$W^{[1]} \in R^{d_{model} \times d_{FF}}$

$W^{[2]} \in R^{d_{FF} \times d_{model}}$

FLOPS per Layer

- Quadratic in both sequence length and embedding dimension

$LN(x_i + FFN(x_i)) = \gamma \cdot \left(\frac{x_i - \mu_i}{\sigma_i}\right) + \beta$

$\sigma_i^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i^{(j)} - \mu_i)^2$

- For smaller sequences
  - FFN is the bottleneck
- For larger sequences
  - Attention layer is the bottleneck

Pre-LN is used because putting LayerNorm *before* the attention/FFN leaves the residual path as a clean identity connection, allowing gradients to flow directly and preventing vanishing/exploding gradients. In Post-LN, gradients must pass through the LayerNorm Jacobian, which disrupts the residual stream and causes instability in deep Transformers.

Also, for LN, training & inference behave the same.

- KV Caching makes attention cost linear in L_{seq}